Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Disaster Resource Allocation System

Team Name: *Team #*	*Disaster Manager* *DSCPP-III-2026-T100*
Team member 1 (Team Lead)	*Bisht, Shubham – 2510012245* *bishtshubham265@gmail.com* 
Team member 2	*Bhandari, Samiksha – 2510010580* *bhandarisamiksha22@gmail.com* 
Team member 3	*Semwal, Sarthak – 2510010180* *sarthaksemwal92@gmail.com* 
Team member 4	*Rawat, Nikhil Singh – 2510012376* *nikhil.singh0072@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

The Disaster Resource Allocation System is a C++ application that helps manage disaster relief work. It keeps a record of the relief requests and the resources available, and then assigns the resources based on how urgent each request is and how close the nearest center is.
When disasters like floods, earthquakes and fires happen, things like food, medicine, shelter and volunteers are mostly sent out by hand. This takes time, and it often goes wrong in another way too, where one area ends up with more than it needs while another area gets very little.In our project we have tried to fix it by taking requests in smart order. Every coming request go to priority queue 1st so the most urgent request is taken first . We represent relied centers and affected zones as weighted graph which helps our system to find the nearest center which enough stock to fulfill the request. The goal is. to reduce time between a request and resources assigned. From Data structures we used graphs , heaps, linked lists and hashing. From OOPS we use Resourse, Volunteer and Request classes , inheritance, polymorphism and STL. In phase 2 we build the modules and worked on making front end and back end side by side.We still have the part of database integration , request module, and testing and training of our system on scenarios .

Updated Project Approach and Architecture (2 pts)

The system follows a modular architecture made up of 3 main components which are a request manager , allocation engine ,resource and volunteer database. Incoming relief requests carrying a type, location and urgency level will be stored in a linked list, based request queue and ordered using a priority queue , so the most important request processed 1st. The allocation engine represent relied centers and affected zone as graph and applies shortest path search to locate the nearest center with sufficient stock . A hash table is used for resourve lookup by ID which keep retrieval fast. On the C++ side, the resources are set up as an inheritance hierarchy (Resource Food, Medical, Shelter). Each type overrides a polymorphic \texttt{allocate()} method, so every resource is allocated in its own way. volunteer and request are kept as separate classes . We used standard template library vector , map , queue for thebringementation and SQLite stores resource inventory , requests , volunteers and allocation logs. When the allocation is done , the results are logged and shown as dispatch list. For the development we followed design,implementation and testing . Each team member is responsible for different modules and bring everything together through a Github repository .

SYSTEM MODULES USED

Module Breakdown

Module	Description	DSA / OOP Concepts Used	Status
Resource Module	Manages inventory of Food, Medical, and Shelter resources; tracks stock levels and handles allocation of stock to requests.	Inheritance, Polymorphism (`allocate()` override), Classes & Objects	**Completed**
Volunteer Module	Registers volunteers, stores their skills and availability, and assigns them to relief tasks.	Classes & Objects, STL `vector`, Encapsulation	**Completed**
Request Module	Captures incoming relief requests (location, type, urgency) and queues them for processing.	Linked List, Queue, Classes	**In Progress**
Allocation Engine	Core matching logic: selects the highest-priority request and finds the nearest relief center with sufficient stock.	Priority Queue (Heap), Graph, Shortest Path Search, Hashing (resource lookup)	**Completed** (design) **In Progress** (integration)
Database Module	Persists resources, requests, volunteers, and allocation logs using SQLite; loads state on start-up.	File I/O, Hashing (ID lookup), SQL table relationships	**Completed**
Dispatch / Reporting Module	Generates the final dispatch list and allocation reports after a request is matched.	STL `map`, Polymorphism, String formatting	**Pending**

Libraries & Tools Used

Library / Tool	Purpose
C++ Standard Template Library (STL)	`vector`, `map`, `priority_queue`, `queue` for core data structures
SQLite3	Lightweight relational database for persisting resources, requests, volunteers, and logs
Git & GitHub	Version control and collaboration across the four-member team
g++ / GCC Compiler	Compilation and build of the C++ codebase
VS Code	Primary development environment
LaTeX (Overleaf)	Preparation of SRS, Phase reports, and documentation

Tasks Completed (7 pts)

Task Completed	Team Member
Project planning and requirement definition	Shubham Bisht
Core matching algorithm design (priority queue + graph search)	Shubham Bisht
Database schema design and table relationships	Samiksha Bhandari
Unit test execution for Resource, Volunteer, queue and graph modules	Samiksha Bhandari
Backend logic for Resource and Volunteer modules	Sarthak Semwal
SRS documentation, slide preparation, and system output formatting	Nikhil Singh Rawat

Challenges/Roadblocks (7 pts)

The main roadblock at this stage is the database integration, which is currently failing. Allocation logs and request records are not being saved correctly to SQLite because some C++ class attributes do not match the table columns, and write operations are not committed properly. To solve this, the team will align the schema with the class definitions, create a single `DatabaseManager` class to handle all connections, and use prepared statements and transactions before re-running the integration tests.

Since four modules are developed in parallel, keeping class interfaces consistent is also challenging; the team uses a shared repository and agreed interfaces to reduce conflicts. Requests with the same urgency are ranked fairly using request time as a tie-breaker, and partial stock at the nearest center is handled by falling back to the next-nearest center and splitting the allocation. These cases will be validated with simulated disaster scenarios.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Fix the failing SQLite integration (schema alignment, commits)	Samiksha Bhandari, Sarthak Semwal
Integrate the matching algorithm with the Request Manager	Shubham Bisht
Complete the Request module and volunteer assignment logic	Sarthak Semwal
System-level testing with simulated disaster scenarios	Samiksha Bhandari
Dispatch-list output, final report, and demo preparation	Nikhil Singh Rawat

Project Outcome/Deliverables (2 pts)

The project will deliver a working C++ prototype that records relief requests, resources, and volunteers, ranks requests by urgency, and allocates the nearest available resources through a graph-based matching algorithm.The main deliverables are the source code, the SQLite database schema, unit-test programs with their results, the SRS document, allocation and dispatch reports, the project slides, and the final documentation. The project brings Data Structures concepts (graphs, heaps, linked lists, hashing) and OOP concepts (classes, inheritance, polymorphism, STL) together in one working system. We also show that it actually helps, by measuring the drop in response time across simulated disaster scenarios.

Progress Overview (2 pts)

We're roughly 55 to 60 percent done with the project.The requirements, architecture, core matching algorithm design and database schema are all finished. The backend modules for Resource and Volunteer are built too, and they pass their unit tests.
\textbf{Ahead of schedule:} The algorithm design and module-level unit testing were both wrapped up early.
\textbf{Behind schedule:} Database integration is running late because the SQLite integration test is still failing. That also holds up system-level testing, since it needs a working database layer.
Next, we'll fix the database integration, finish the Request module, bring in the allocation engine, and run full scenario-based tests.

Codebase Information (2 pts)

Repository: `https://github.com/Ixotic27/disaster-relief-system`

Branch: main

Important commits / milestones:
– Initial project structure and class skeletons
– Resource class hierarchy (Food, Medical, Shelter) and Volunteer module
– Priority queue and graph-based matching algorithm design
– SQLite database schema and table relationships
– Unit test programs for core modules

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit Test – Resource class	**PASS**	Add, update and deduct stock verified for Food, Medical and Shelter; `allocate()` returns correct quantities.
Unit Test – Volunteer class	**PASS**	Registration, skill tagging and availability toggling work as expected.
Unit Test – Priority queue	**PASS**	Requests served in urgency order; equal urgency resolved by request time.
Unit Test – Graph shortest path	**PASS**	Nearest relief center correctly found on a 10-node sample map.
Integration Test – SQLite database	**PASS**	Allocation logs not saved: class attributes do not match table columns and writes are not committed. Fix in progress.
System Test – Simulated disaster scenario	**NOT RUN**	Depends on database integration; scheduled after the database fix.

Deliverables Progress (2 pts)

Completed: Requirements and SRS; system architecture; core matching algorithm design; database schema; Resource and Volunteer backend modules; unit tests for core modules; project slides and output formatting.

In Progress: SQLite database integration (integration test currently failing, fix underway); Phase-II documentation.

Pending: Request module completion; allocation engine integration; system-level testing with simulated scenarios; final report and demonstration.